Q1 (a)

Liveness. Termination means that P will make progress and won't deadlock. Termination is the good thing of interest.

(b)

Safety. No null pointer dereferences means that each time a value is expected in a program, some value will be present there. This also means no runtime errors. Bad thing of interest are null pointer dereferences.

(c)

Liveness because the program always terminates and returns an even number. Good thing of interest - termination and returning an even number.

(d)

Liveness. Good thing of interest - winning a Turing Prize.

(e)

Safety. Bad thing of interest - not winning the prize by 2030.

Q2 (a)

Yes. Consider the following ordering:
x=0
y=0
y=1
a=y // 1
b=x // 0
x=1
PSO allows write-write reordering on different locations, hence y=1 can happen before x=1. At the end, a=1, b=0, as required.

(b)

Yes. Consider an ordering similar to the one in (a).
x=0
y=0
y=1
a=y // 1
b=x // 0
x=1
sfence
Sfence doesn't impose any reordering constraints on write-write reordering. It only influences non-temporal stores, which aren't present in the fragment of code in question.

(c)

Coursework 2

09.11.2022

Q2.

(a) (c)

Memory write:

$$\frac{B(T) = b \quad b' = b_1 \cdot (W, x, v) \cdot b_2 \quad B' = B[T \mapsto b']}{M, B \xrightarrow{T:(W,x,v)}_m M, B'}$$

Memory read:

$$\frac{B(T) = b \quad get(M, b, x) = v}{M, B \xrightarrow{T:(R,x,v)}_m M, B}$$

where $get(M, b, x)$ defined as in slides for TSO

Memory fence:

$$\frac{B(T) = \emptyset}{M, B \xrightarrow{T : \mathrm{MF}}_{c} M, B}$$

Successful RMWs:

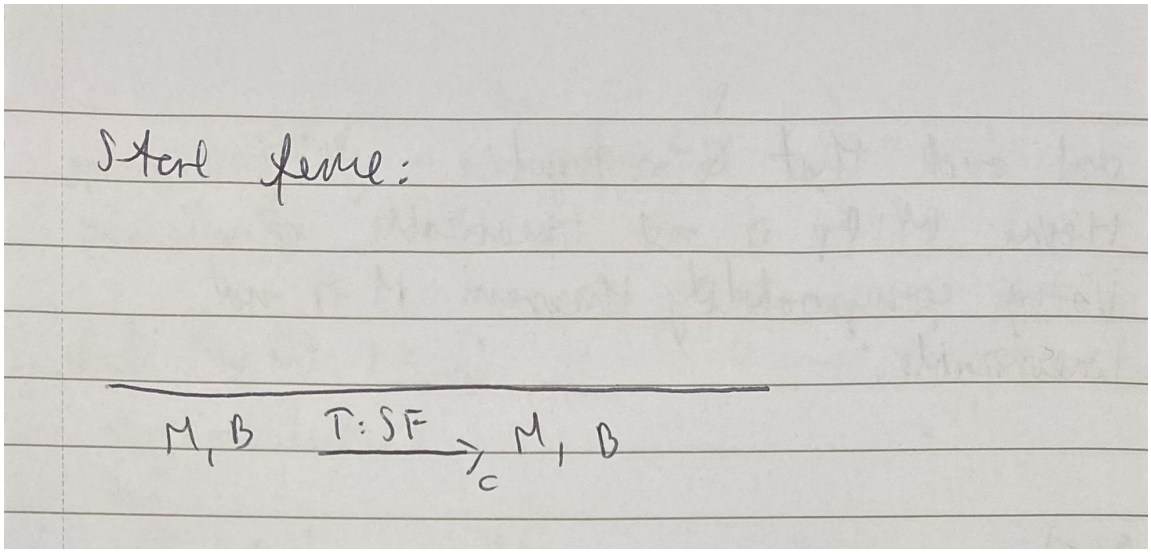
$$\frac{B(T) = \emptyset \quad M(x) = v_o \quad M' = M[x \mapsto v_n]}{M, B \xrightarrow{T : (U, x, v_o, v_n)}_{m} M', B}$$

Failed RMWs:

$$\frac{B(T) = \emptyset \quad M(x) = v}{M, B \xrightarrow{T : (U, x, v, \bot)}_{m} M, B}$$

Propagation of delayed writes:

$$\frac{B(T) = (W, x, v) \cdot b \quad M' = M[x \mapsto v] \quad B' = B[T \to b]}{M, B \xrightarrow{T : t}_{m} M', B'}$$



Q3 (a) (i)

3.

(a)

(i) Consider the histories of p and q seperately:

p:

$H|_p = B: p.deq()$

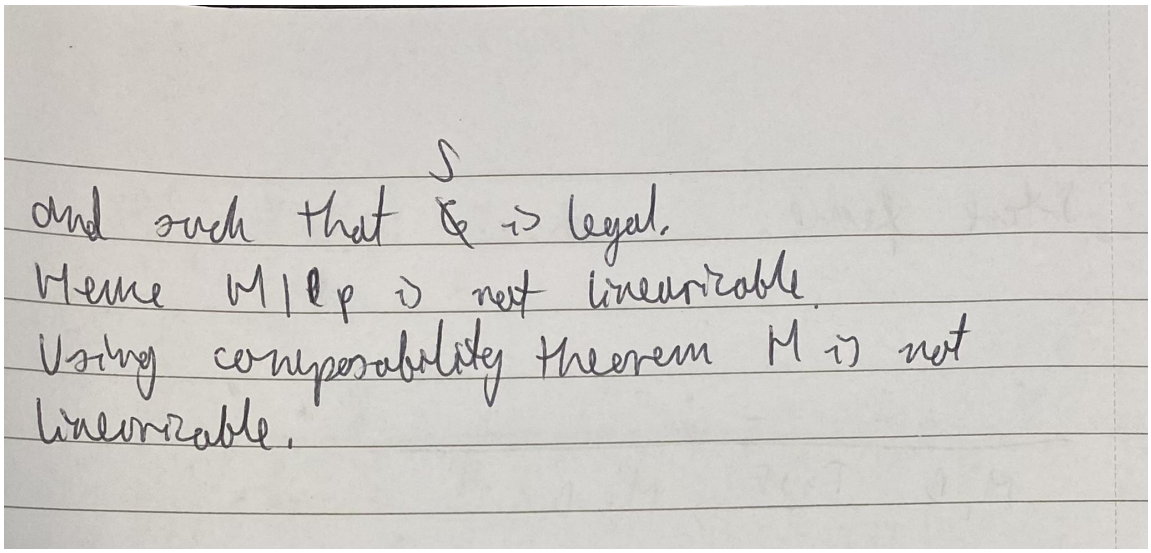
$B: \not{\emptyset}\ p:?$

$A: p.enq(1)$

$A: p: void$

$\rightarrow_{H|_p} = \{B: p:? \rightarrow A: p.enq(?)\}$

There doesn't exist an equivalent history $S \in |p$ such that $\not{\emptyset} \rightarrow_{H|p} C \rightarrow_{S \in |p}$



(ii)

Yes.

Original history H:

B: p.deq()
B: p: 1
A: p.enq(1)
A: p: void
B: q.enq(2)
B: q: void
A: q.enq(1)
A: q: void
B: q.deq()
B: q: 1

Equivalent history S:

A: p.enq(1)
A: p: void
B: p.deq()
B: p: 1
A: q.enq(1)
A: q: void
B: q.enq(2)
B: q: void
B: q.deq()
B: q: 1

For the equivalent history S we have the following order. For p - enq(1), deq(2), which is valid. For q - enq(1), enq(2), deq(1) - also valid. S doesn't need to preserve the order present in H. Hence H is sequentially consistent.

(b)

Original history H':

A: p.enq(1)
B: q.deq()
A: p: void
B: q: 1
A: p.enq(2)
A: p: void
A: q.enq(1)
B: p.deq()
A: q: void
B: p: 2

Per-thread history H' | A:

A: p.enq(1)
A: p: void
A: p.enq(2)
A: p: void
A: q.enq(1)
A: q: void

Per-thread history H' | B:

B: q.deq()
B: q: 1
B: p.deq()
B: p: 2

Equivalent history S:

A: p.enq(1)
A: p: void
A: p.enq(2)
A: p: void
A: q.enq(1)
A: q: void
B: q.deq()
B: q: 1
B: p.deq()
B: p: 2

The equivalent history S doesn't need to preserve order from H'. S is legal wrt. to p and q. Hence H' is sequentially consistent.

(c)

Yes. Intuitively, you can project all the events on a single line, so that they make sense.

History H'':

A: p.enq(17)
A: p: void
B: p.size()
C: p.enq(23)
C: p: void
A: p.enq(42)
B: p: 2
B: p.size()
B: p: 3

History G - after appending pending method response:

A: p.enq(17)
A: p: void
B: p.size()
C: p.enq(23)
C: p: void
A: p.enq(42)
B: p: 2
B: p.size()
B: p: 3
A: p: void

G|A:

A: p.enq(17)
A: p: void
A: p.enq(42)
A: p: void

G|B:

B: p.size()
B: p: 2
B: p.size()
B: p: 3

G|C:

C: p.enq(23)
C: p: void

$\rightarrow_G$ = {

A: p.enq(17) → B: p.size(2),
A: p.enq(17) → C: p.enq(23),
A: p.enq(17) → A: p.enq(42),
B: p.size(2) → B: p.size(3),
C: p.enq(23) → B: p.size(3),
C: p.enq(23) → A: p.enq(42)

}$^+$

Consider an equivalent history S:

A: p.enq(17)
A: p: void
C: p.enq(23)
C: p: void
B: p.size()
B: p: 2
A: p.enq(42)
A: p: void
B: p.size()
B: p: 3

$\rightarrow_S$ is a subset of $\rightarrow_G$ and S is a legal history. Hence H'' is linearizable.